Lab Manual: Day 4 – Refactoring Callback Hell to Async/Await

Instructor: Kashif Raza
Duration: 45 Minutes
Difficulty: Intermediate

🎯 Objective

In this lab, you will simulate a real-world E-Commerce checkout flow. You will start with a legacy "Callback Hell" implementation, refactor it using Promises, and finally modernize it using `async/await` to achieve linear control flow and centralized error handling.

🛠️ Prerequisites

- Node.js installed (v14+)
- Code Editor (VS Code recommended)
- Terminal access

📜 Scenario: The E-Commerce Checkout Pipeline

We are building the backend logic for an online store. The checkout process requires three sequential API calls:

1. **Check Inventory:** Verify the item is in stock.
2. **Process Payment:** Charge the customer's card.
3. **Initiate Shipping:** Send order details to the logistics provider.

Constraint: Step 2 cannot happen until Step 1 succeeds. Step 3 cannot happen until Step 2 succeeds.

Step 1: Setup & The "Legacy" Code

Create a new file named `checkout.js`. Copy and paste the following code. This represents the "Before" state—deeply nested callbacks (Callback Hell).

```javascript
// --- Simulated APIs (Do Not Modify) ---
const inventoryAPI = (item, callback) => {
  setTimeout(() => {
    if (item === "laptop") callback(null, { id: 101, stock: true });
    else callback("Item not found");
  }, 1000);
};

const paymentAPI = (orderData, callback) => {
  setTimeout(() => {
    if (orderData.stock) callback(null, { success: true, txnId: "TXN_999" });
    else callback("Payment failed: Insufficient stock");
  }, 1000);
};

const shippingAPI = (orderDetails, callback) => {
  setTimeout(() => {
    if (orderDetails.success) callback(null, "Shipped via Express");
    else callback("Shipping failed");
  }, 1000);
};

// --- TODO: Fix the function below ---
function checkoutLegacy(item) {
  console.log("Starting Checkout...");

  inventoryAPI(item, (err, inventoryResult) => {
    if (err) console.error("Error:", err);
    else {
      console.log("Inventory Check Passed");
      paymentAPI(inventoryResult, (err, paymentResult) => {
        if (err) console.error("Error:", err);
        else {
          console.log("Payment Processed");
          shippingAPI(paymentResult, (err, shippingResult) => {
            if (err) console.error("Error:", err);
            else console.log("Success:", shippingResult);
          });
        }
      });
    }
  });
}
```

```
// Run the legacy code
// checkoutLegacy("laptop");
```

Task:

1. Uncomment the line `checkoutLegacy("laptop");` and run the file (`node checkout.js`).
2. Observe the "Pyramid of Doom" indentation.
3. Discuss: Where would you add a `try/catch` here? (Answer: You can't easily; you have to check `if (err)` at every single level).

Step 2: The Promise Refactor (The "Bridge")

Delete the `checkoutLegacy` function. We will now use Promises to flatten the logic.

Task:

1. Wrap the simulated APIs to return Promises (or use `util.promisify` , but for this lab, let's rewrite one wrapper function to understand the mechanism).
2. Rewrite `checkoutPromises` using `.then()` chains.

```
// Solution Template
function checkoutPromises(item) {
  // 1. Convert Callback API to Promise
  const checkInventory = (item) => {
    return new Promise((resolve, reject) => {
      inventoryAPI(item, (err, res) => err ? reject(err) : resolve(res));
    });
  };

  // 2. Chain the calls
  checkInventory(item)
    .then((invRes) => {
      console.log("Inventory OK");
      // Return next promise to chain
      return new Promise((resolve, reject) => {
        paymentAPI(invRes, (err, res) => err ? reject(err) : resolve(res));
      });
    })
    .then((payRes) => {
      console.log("Payment OK");
      return new Promise((resolve, reject) => {
        shippingAPI(payRes, (err, res) => err ? reject(err) : resolve(res));
      });
```

```
    })
    .then((shipRes) => {
      console.log("Success:", shipRes);
    })
    .catch((err) => {
      console.error("Unified Catch:", err);
    });
}
```

Run: Call `checkoutPromises("laptop")`. Notice how the indentation resets to the left edge.

Step 3: The Final Refactor (Async/Await)

This is the industry standard. Delete the code from Step 2 and write the following.

Task:

1. Create an `async` function named `checkoutModern`.
2. Use `await` for each step.
3. Wrap the entire block in a `try...catch`.

```
// --- Lab Solution ---
async function checkoutModern(item) {
  try {
    console.log(`[Start] Processing order for ${item}`);

    // Step 1: Await Inventory
    const inventory = await new Promise((resolve, reject) => {
      inventoryAPI(item, (err, res) => err ? reject(err) : resolve(res));
    });
    console.log("✅ Inventory Checked");

    // Step 2: Await Payment
    const payment = await new Promise((resolve, reject) => {
      paymentAPI(inventory, (err, res) => err ? reject(err) : resolve(res));
    });
    console.log("✅ Payment Received");

    // Step 3: Await Shipping
    const shipping = await new Promise((resolve, reject) => {
      shippingAPI(payment, (err, res) => err ? reject(err) : resolve(res));
    });
    console.log("✅ Order Shipped");
```

```
    return shipping;

  } catch (error) {
    // Centralized Error Handling
    console.error("❌ Critical Error:", error);
    // In production, you might trigger a rollback here
  }
}

// Run the modern code
checkoutModern("laptop");
```

🧪 Step 4: Validation & Verification

Run your final script. Compare the results.

Expected Output:

```
[Start] Processing order for laptop
✅ Inventory Checked
✅ Payment Received
✅ Order Shipped
```

Test the Error Path:
Change the input to `checkoutModern("phone");`.
Expected Output:

```
[Start] Processing order for phone
❌ Critical Error: Item not found
```

🏆 Bonus Challenge (Parallel Execution)

Scenario: While checking out, we want to fetch "User Reviews" and "Similar Products" at the same time to render the confirmation page faster. They are independent of the checkout flow.

Task:
Use `Promise.all` inside your async function to fetch these two items simultaneously.

```
// Hint
const [reviews, similar] = await Promise.all([
  fetchReviews(item),
  fetchSimilarItems(item)
]);
```